Folha de Respostas -Aula 7

Nome do Aluno: Felipe Uehara Gondo **No. USP: 1368061**

Instruções: A) Com referência à apostila da parte prática (seguindo a enumeração), completar os campos abaixo com apenas as informações e dados solicitados. B) Para arquivos a subir, denomine-os pela enumeração do item.

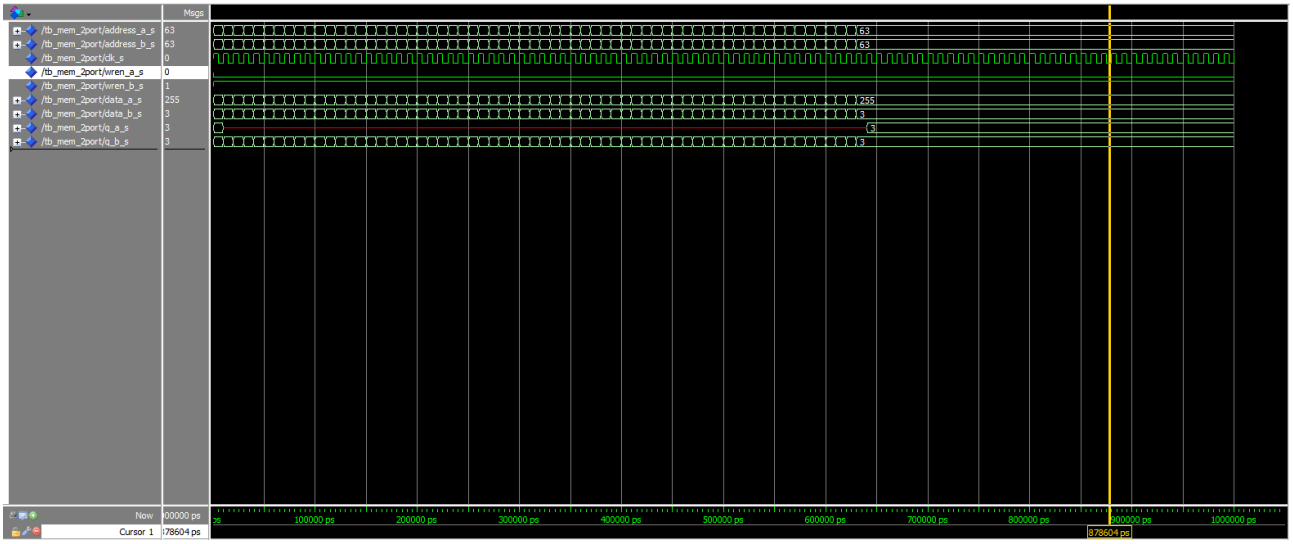
2) Mem

f) Insira no quadro abaixo o novo processo dentro do módulo stimuli module para os **corner cases 2.a e 2.b**, centrados no endereço N1 calculado para a provinha. Explicite as mudanças realizadas.

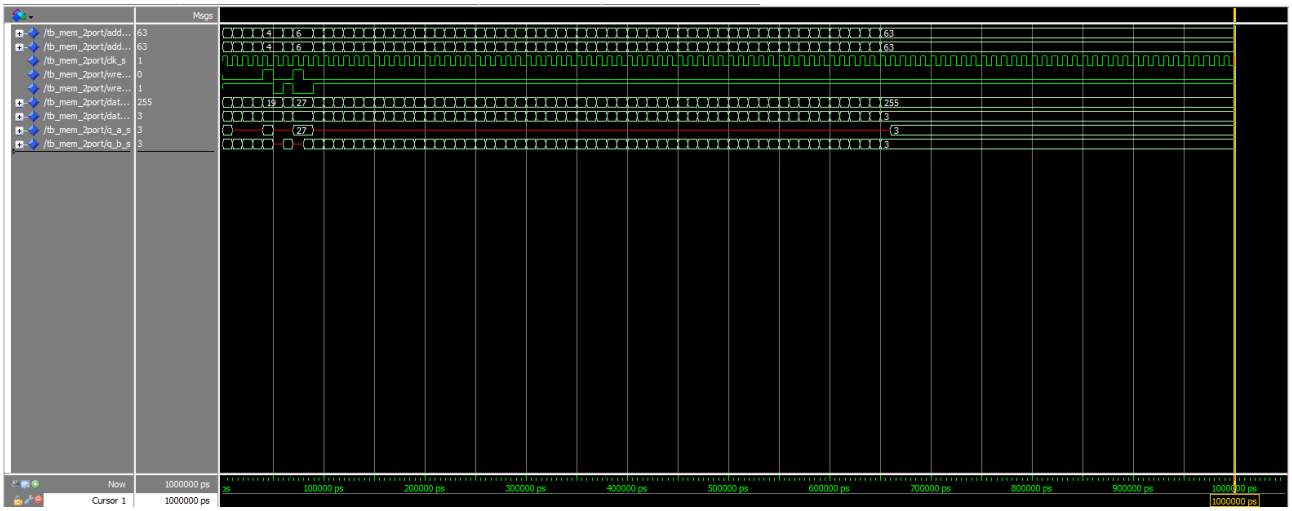
```
for i in 0 to 63 loop
            if i = 4 then --2)a. Escrita de dados distintos no mesmo endereço
                check_mem(  std_logic_vector(to_unsigned(i, 6)), --addess_a
                            std_logic_vector(to_unsigned(i, 6)), --address_b
                            '1',                                 --wren_a
                            '1',                                 --wren_b
                            std_logic_vector(to_unsigned(4*i+3, 8)), --data_a
                            std_logic_vector(to_unsigned(255-4*i, 8))); --data_b

                check_mem(  std_logic_vector(to_unsigned(i, 6)), --addess_a
                            std_logic_vector(to_unsigned(i, 6)), --address_b
                            '0',                                 --wren_a
                            '0',                                 --wren_b
                            std_logic_vector(to_unsigned(4*i+3, 8)), --data_a
                            std_logic_vector(to_unsigned(255-4*i, 8))); --data_b
            end if;

            if i = 6 then--2)b. Escrita no porto A e Leitura no porto B
                check_mem(  std_logic_vector(to_unsigned(i, 6)), --addess_a
                            std_logic_vector(to_unsigned(i, 6)), --address_b
                            '1',                                 --wren_a
                            '0',                                 --wren_b
                            std_logic_vector(to_unsigned(4*i+3, 8)), --data_a
                            std_logic_vector(to_unsigned(255-4*i, 8))); --data_b

                check_mem(  std_logic_vector(to_unsigned(i, 6)), --addess_a
                            std_logic_vector(to_unsigned(i, 6)), --address_b
                            '0',                                 --wren_a
                            '0',                                 --wren_b
```

```
                        std_logic_vector(to_unsigned(4*i+3, 8)), --data_a
                        std_logic_vector(to_unsigned(255-4*i, 8))); --data_b
            end if;

            if i /=4 and i/=6 then --2)b. Escrita no porto B e Leitura no porto
A
                check_mem(  std_logic_vector(to_unsigned(i, 6)), --addess_a
                            std_logic_vector(to_unsigned(i, 6)), --address_b
                            '0',                                 --wren_a
                            '1',                                 --wren_b
                            std_logic_vector(to_unsigned(4*i+3, 8)), --data_a
                            std_logic_vector(to_unsigned(255-4*i, 8))); --data_b
            end if;
        end loop;
```

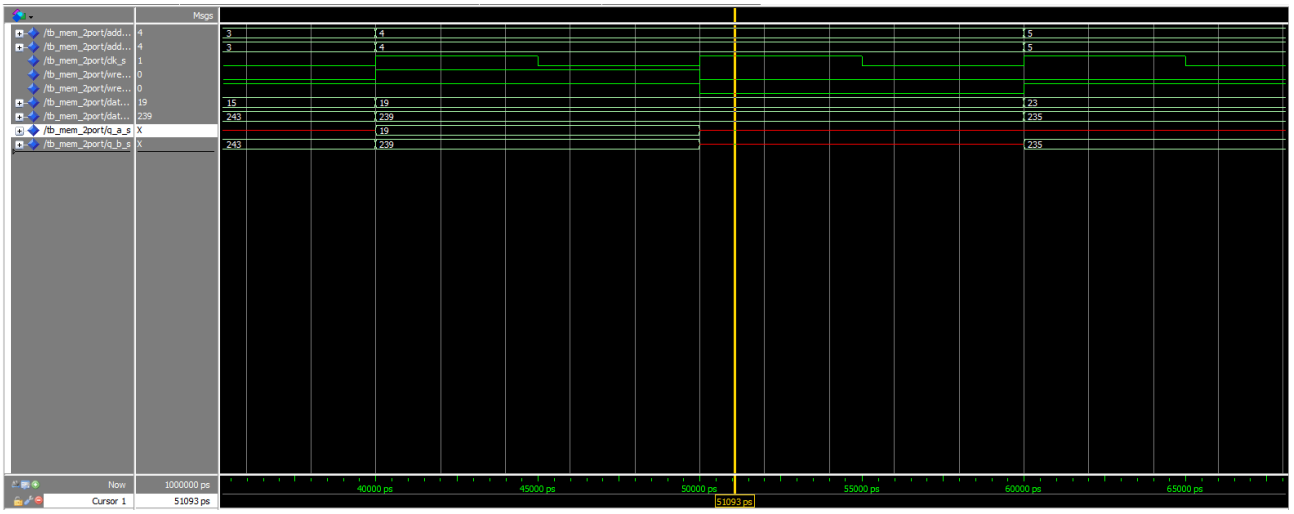
g.1) Faça o upload de imagem(ns) com a simulação <u>com os corner cases.</u> Identifique na imagem pelo momento/tempo do evento



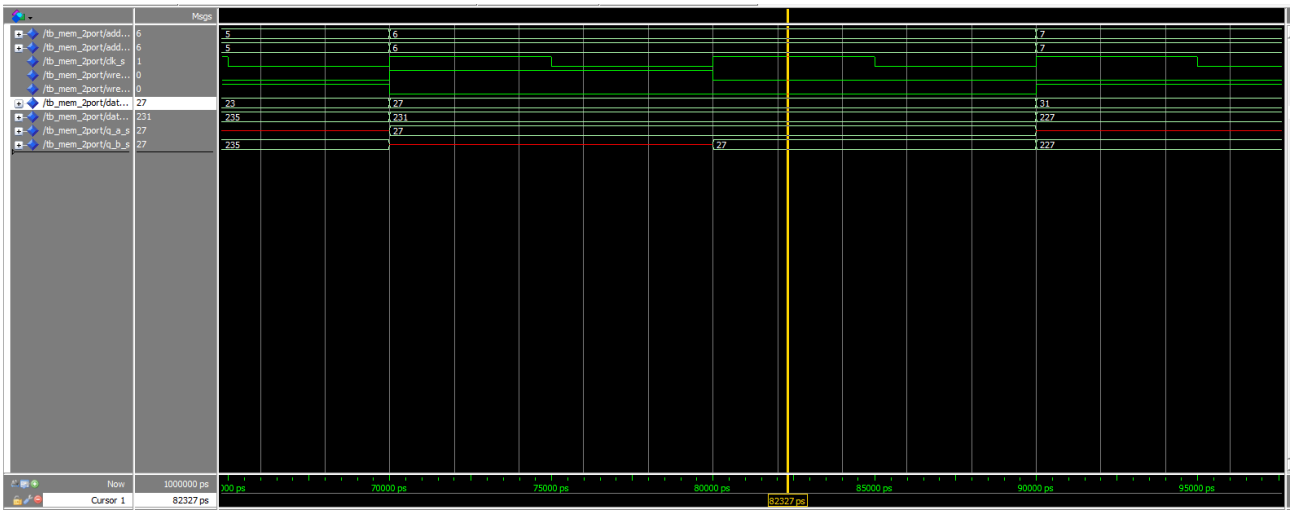
Simulação Original sem os corner cases



Simulação com os corner cases considerados



Simulação 2)a no endereço 4



Simulação 2)b no endereço 6

g.2) Explique como se pode verificar que todas as condições solicitadas na imagem ocorreram.

Segundo a apostila conceitual desta aula, quando se habilita a escrita simultânea de dados distintos no mesmo endereço, o valor lido no endereço será “X” no bloco da Intel durante o ciclo de leitura, já no ciclo de escrita, os valores amostrados serão os dados de escrita de cada porto do bloco em si. Para a simulação quando há escrita no porto A e leitura no porto B, o valor amostrado em B durante a escrita será “X” e o valor anterior (no caso, 27) durante o ciclo de leitura.

3)

a) Insira no quadro abaixo a arquitetura do **base_mem_simple** por você completada.

```
mem : mem_2port
    port map
    (
```

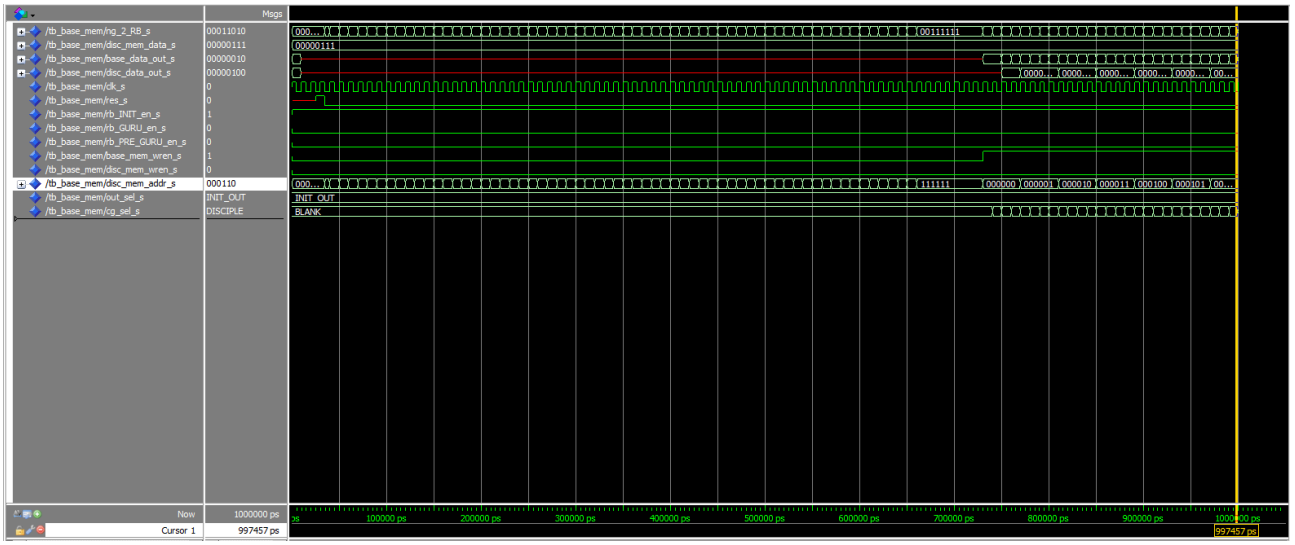
```
        address_a => base_2_mem_s.mem_a_addr,
        address_b => disc_2_mem_s.mem_b_addr,
        clock     => clk_s,
        data_a    => base_2_mem_s.data_a,
        data_b    => disc_2_mem_s.data_b,
        wren_a    => base_2_mem_s.mem_wr_en,
        wren_b    => disc_2_mem_s.mem_wr_en,
        q_a       => base_data_out,
        q_b       => disc_data_out
    );

    rb : reg_bank
        generic map
        (
            WIDTH => WIDTH
        )
        port map
        (
            clk            => clk_s,
            res            => res_s,
            ng_2_RB        => ng_2_RB,
            load_INIT      => rb_INIT_en,
            load_GURU      => rb_GURU_en,
            load_PRE_GURU  => rb_PRE_GURU_en,
            out_sel        => out_sel,
            guru_addr      => guru_addr_s,
            guru_prev_addr => guru_prev_addr_s,
            rb_out         => rb_2_alu_s
        );

    c_g : code_gen
        port map
        (
            ctrl_code_sel => cg_sel,
            mem_code_w    => mem_a_data_s
        );
```

o, p) Insira no quadro o trecho do stimuli module modificado; os corner cases devem ser centrados no endereço N1 calculado para a provinha.

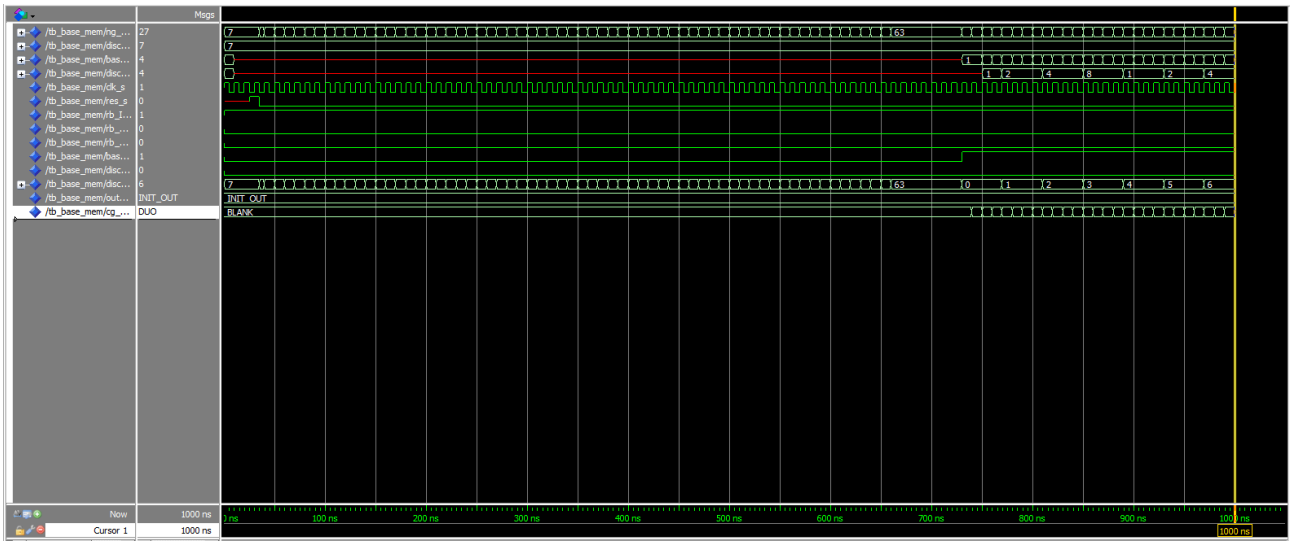
```
    -- disciple write enable for all positions of memory
```

```
        for i in 0 to 63 loop

            -- cycle of write for disc (i+2 i position i

            check_mem_base_ctrl('1', '0', '0', '0', INIT_OUT, BLANK);    --
control values
            check_mem_base_datapath(std_logic_vector(to_unsigned(i, 8)));  --
address to registers input
            check_mem_disc('1', std_logic_vector(to_unsigned(i, 6)),
std_logic_vector(to_unsigned(i+2, 8)));    -- any data from disc

            wait until rising_edge (clk_s);

            -- cycles of read for disc

            check_mem_base_ctrl('1', '0', '0', '0', INIT_OUT, BLANK);    --
control values
            check_mem_base_datapath(std_logic_vector(to_unsigned(i, 8)));  --
address to registers input
            check_mem_disc('0', std_logic_vector(to_unsigned(i, 6)),
std_logic_vector(to_unsigned(i+2, 8)));    -- any data from disc

        end loop;

        wait;
    end process sim;
end architecture test;
```

q.) Faça o upload de imagem(ns) com a simulação com as situações de teste por você providenciado. Identifique na figura estas situações.



Simulação original



Simulação Pós alterações